

B2Com – Dialpad externo

La conexión de voz con el backend se basa en 3 componentes:

- Conexión socket.io, se utiliza para la recepción de eventos del lado del servidor.
- Endpoint API en el backend, para enviar comando al servidor.
- Librería <https://jssip.net/> (en oposición a SipJS que se venía utilizando en versiones anteriores)

Url del backend:

Las peticiones al backend se realizan por POST a:

<https://<servidor>.vozipcenter.com/...>

Sustituyendo <servidor> por el domino del cliente asociado a b2com de la instalación que corresponda.

La ruta después del dominio es distinta entre la función de login y de usuario ya autenticado.

Autenticación:

Se puede realizar a través de POST a la siguiente URL:

<https://<servidor>.vozipcenter.com/l/0/v3/backend>

La letra L (de login) después del domino, indica que estamos realizando una llamada al namespace de login, por lo tanto, sin autenticar.

El número 0 (cero) sirve para mantener múltiples sesiones en un mismo navegador, pero por ahora está en pruebas.

Enviando en el body:

```
{ "method": "agente/login", "args": { "keepOpen": true, "u": "101", "p": "j1" } }
```

La respuesta, nos establecerá 2 cookies en el navegador, que serán las encargadas de autorizar el resto de peticiones.

La respuesta será:

```
{  
  "success": 1,  
  "orden": 0,  
  "rol": "agent"  
}
```

El “orden” proporcionado, corresponde con el de la URL, y el que deberíamos utilizar en las llamadas posteriores al backend del usuario.

En caso de error devolverá:

```
{  
  "message": "Datos incorrectos.",  
  "success": 0  
}
```

Acceso a los datos del usuario:

Una vez autenticado, el acceso a los datos del usuario, se realiza a través de la inyección del script:

`https:// <servidor>.vozzipcenter.com/u/0/sys/profile.js`

Una vez inyectado dicho script en el navegador, podemos acceder a los objetos:

`window.__b2com_state`

Contiene información que permanece inalterable durante toda la vida de la página.

`window.__b2com_realtime`

Contiene información susceptible de ser modificada desde el backend, a través de los eventos de sincronización.

Por ahora, dichos objetos contienen múltiple información del usuario de distintos módulos (llamadas, chats, whatsapp, contactos, etc). En el futuro limitaremos según el origen del acceso más o menos información.

Para el Dialpad en Salesforce, por ahora nos interesa, el objeto: `window.__b2com_state.user`:

```

1 {
2   "user": {
3     "id": 30,
4     "token": "7f4244d0-09af-406d-81a8-be873255fd9d",
5     "nombre": "J101",
6     "grupos": [
7       {
8         "id": 85,
9         "titulo": "CALL FAUZA",
10        "callerid": [],
11        "url": "https://dev-jorge3.vozipcenter.com/s/ge1f-3266"
12      },
13      {
14        "id": 84,
15        "titulo": "ENCOIN",
16        "callerid": [],
17        "url": null
18      },
19      {
20        "id": 1,
21        "titulo": "grupo A",
22        "callerid": [
23          "912903122",
24          "942049038",
25          "910311447",
26          "912903122",
27          "942049038",
28          "910311447"
29        ],
30        "url": "https://dev-jorge3.vozipcenter.com/s/g98c-6f2c"
31      },
32      {
33        "id": 85,
34        "titulo": "CALL FAUZA",
35        "callerid": [],
36        "url": "https://dev-jorge3.vozipcenter.com/s/ge1f-3266"
37      }
38    ],
39     "estados": [
40       {
41         "id": 1,
42         "titulo": "Estado 1",
43         "url": null
44       },
45       {
46         "id": 2,
47         "titulo": "Estado 2",
48         "url": null
49       }
50     ],
51     "agentes": [
52       {
53         "id": 1,
54         "nombre": "Agente 1",
55         "telefono": "912903122",
56         "url": null
57       },
58       {
59         "id": 2,
60         "nombre": "Agente 2",
61         "telefono": "942049038",
62         "url": null
63       }
64     ],
65     "avatar": "",
66     "blank_profile": "",
67     "id_centralita": "13220",
68     "modulo_whatsapp": 1,
69     "isVistaDetallada": 0,
70     "permisos": [
71       {
72         "id": 1,
73         "nombre": "Permiso 1",
74         "url": null
75       },
76       {
77         "id": 2,
78         "nombre": "Permiso 2",
79         "url": null
80       }
81     ],
82     "causas": [
83       {
84         "id": 1,
85         "nombre": "Causa 1",
86         "url": null
87       },
88       {
89         "id": 2,
90         "nombre": "Causa 2",
91         "url": null
92       }
93     ],
94     "extension_ip": "310531101",
95     "ocultar_contador_whatsapp": 1,
96     "minutos_wa": 2,
97     "google": {
98       "id": 1,
99       "nombre": "Google 1",
100      "url": null
101    },
102     "whatsapp": {
103       "id": 1,
104       "nombre": "WhatsApp 1",
105       "url": null
106     },
107     "contactos": [
108       {
109         "id": 1,
110         "nombre": "Contacto 1",
111         "url": null
112       },
113       {
114         "id": 2,
115         "nombre": "Contacto 2",
116         "url": null
117       }
118     ],
119     "llamadas": [
120       {
121         "id": 1,
122         "nombre": "Llamada 1",
123         "url": null
124       },
125       {
126         "id": 2,
127         "nombre": "Llamada 2",
128         "url": null
129       }
130     ],
131     "conversations": [
132       {
133         "id": 1,
134         "nombre": "Conversación 1",
135         "url": null
136       },
137       {
138         "id": 2,
139         "nombre": "Conversación 2",
140         "url": null
141       }
142     ]
143   }
144 }

```

Aquí podemos obtener:

Nombre, Listado de grupos, listado de estados, avatar, ...

Listado de grupos:

```

1 {
2   "grupos": [
3     {
4       "id": 85,
5       "titulo": "CALL FAUZA",
6       "callerid": [],
7       "url": "https://dev-jorge3.vozipcenter.com/s/ge1f-3266"
8     },
9     {
10      "id": 84,
11      "titulo": "ENCOIN",
12      "callerid": [],
13      "url": null
14    },
15    {
16      "id": 1,
17      "titulo": "grupo A",
18      "callerid": [
19        "912903122",
20        "942049038",
21        "910311447",
22        "912903122",
23        "942049038",
24        "910311447"
25      ],
26      "url": "https://dev-jorge3.vozipcenter.com/s/g98c-6f2c"
27    },
28    {
29      "id": 85,
30      "titulo": "CALL FAUZA",
31      "callerid": [],
32      "url": "https://dev-jorge3.vozipcenter.com/s/ge1f-3266"
33    }
34  ],
35  "estados": [
36    {
37      "id": 1,
38      "titulo": "Estado 1",
39      "url": null
40    },
41    {
42      "id": 2,
43      "titulo": "Estado 2",
44      "url": null
45    }
46  ],
47  "agentes": [
48    {
49      "id": 1,
50      "nombre": "Agente 1",
51      "telefono": "912903122",
52      "url": null
53    },
54    {
55      "id": 2,
56      "nombre": "Agente 2",
57      "telefono": "942049038",
58      "url": null
59    }
60  ],
61  "avatar": "",
62  "blank_profile": "",
63  "id_centralita": "13220",
64  "modulo_whatsapp": 1,
65  "isVistaDetallada": 0,
66  "permisos": [
67    {
68      "id": 1,
69      "nombre": "Permiso 1",
70      "url": null
71    },
72    {
73      "id": 2,
74      "nombre": "Permiso 2",
75      "url": null
76    }
77  ],
78  "causas": [
79    {
80      "id": 1,
81      "nombre": "Causa 1",
82      "url": null
83    },
84    {
85      "id": 2,
86      "nombre": "Causa 2",
87      "url": null
88    }
89  ],
90  "extension_ip": "310531101",
91  "ocultar_contador_whatsapp": 1,
92  "minutos_wa": 2,
93  "google": {
94    "id": 1,
95    "nombre": "Google 1",
96    "url": null
97  },
98  "whatsapp": {
99    "id": 1,
100   "nombre": "WhatsApp 1",
101   "url": null
102 },
103  "contactos": [
104    {
105      "id": 1,
106      "nombre": "Contacto 1",
107      "url": null
108    },
109    {
110      "id": 2,
111      "nombre": "Contacto 2",
112      "url": null
113    }
114  ],
115  "llamadas": [
116    {
117      "id": 1,
118      "nombre": "Llamada 1",
119      "url": null
120    },
121    {
122      "id": 2,
123      "nombre": "Llamada 2",
124      "url": null
125    }
126  ],
127  "conversations": [
128    {
129      "id": 1,
130      "nombre": "Conversación 1",
131      "url": null
132    },
133    {
134      "id": 2,
135      "nombre": "Conversación 2",
136      "url": null
137    }
138  ]
139 }

```

Lista de estados:

```
{
  "estados": [
    {
      "id": null,
      "descripcion": "Disponible"
    },
    {
      "id": 6,
      "descripcion": "Descanso"
    },
    {
      "id": 9,
      "descripcion": "Reuniones"
    },
    {
      "id": 11,
      "descripcion": "Nuevo Motivo"
    }
  ],
  "agentes": [
```

Del objeto `window.__b2com_realtime.user` obtenemos información del usuario, en el momento de la carga:

```
{
  "user": {
    "grupo_principal": 1,
    "callerid": "912903122",
    "grupo_principal_mobile": 1,
    "estado": {
      "disponible": false,
      "id": 0,
      "descripcion": "DESCONEXIÓN"
    },
    "motivo_web": 0,
    "motivo_mobile": 0,
    "extensiones": [
    ],
  },
  "whatsapp": {
    "call": {
```

De aquí obtenemos el `grupo_principal` del usuario, el `callerid` de dicho grupo, y su estado actual.

Conexión socket.io

La conexión `socket.io` sirve para recibir eventos del servidor, cuando hay algún cambio en alguna de las propiedades del objeto `__b2com_realtime` o si debe notificar al usuario de alguna acción.

Usamos la librería: `"socket.io-client": "^4.7.2"`,

Se puede incluir como javascript del CDN:

<https://cdn.jsdelivr.net/npm/socket.io-client@4.7.2/dist/socket.io.min.js>

Inicializamos la librería con las propiedades:

```
const socket = io({
  closeOnBeforeunload: true,
  transports: ['websocket'],
  query: { orden: '/u/0/' },
});
```

Escuchamos el evento “accepted”, dicho evento se produce cada que vez se reestablece la comunicación. Puede ser al entrar por primera vez, o si ha habido un corte en la conexión y se ha reconecado. Nos envía un parámetro con el objeto “__b2com_realtime” para sustituirlo. Esto permite actualizar la sincronización si el usuario ha estado desconectado varios segundos (si por ejemplo se ha desconectado el wifi y he empezado a hablar con la app móvil, al reconectar, el realtime pasará a indicar que estás ocupado, y se mostrará así en la web).

```
socket.on('accepted', (payload) => {
  console.log('SOCKET.IO ACEPTADO!', payload);
  window.__b2com_realtime = payload;
});
```

Eventos realtime:

Puede llegar un evento “realtime_sync” que trae una serie de transformaciones que debemos realizar en el objeto realtime, de esa forma, cada vez que se modifica en otra sesión alguna de las propiedades, se mantiene sincronizado entre terminales:

Llega un array de cambios, donde cada cambio tiene las siguientes propiedades:

- store: Indica qué propiedad del realtime se modifica (user, whatsapp, call...)
- operation: assing, append, delete, insert ó remove.
- value: depende de la operación
- payload: depende de la operación

Dejo el script que usamos actualmente para esta lógica:

```
socket.on('realtime_sync', (changes) => {
  for (let change of changes) {
    if (change.operation === 'assign') {
      stores[change.store].realtime[change.property][change.payload] =
        change.value;
    } else if (change.operation === 'append') {
      stores[change.store].realtime[change.property].push(change.value);
    } else if (change.operation === 'delete') {

```

```

        delete stores[change.store].realtime[change.property][change.payload];
    } else if (change.operation == 'insert') {
        stores[change.store].realtime[change.property].splice(change.payload, 0, change.value);
    } else if (change.operation == 'remove') {
        stores[change.store].realtime[change.property].splice(change.payload, 1);
    } else {
        stores[change.store].realtime[change.property] = change.value;
    }
}
});

```

El otro evento es “realtime_emit”, que dispara la ejecución de un método en el cliente para distintas lógicas. En vez de utilizar distintos eventos socket.io, se usa uno solo, que proporciona el nombre del método a llamar.

Cada evento, contiene las siguientes propiedades:

- store: Define si es un evento relacionado con “call”, “whatsapp”, “conversations”...
- method: El nombre del método en sí.
- payload: contenido enviado al método a ejecutar.
- cb: si no es null, indica que debemos enviar un ACK al servidor. Dado que estos eventos se disparan en múltiples agentes simultáneamente, cada uno de ellos deberá enviar su ACK y el servidor mantiene un tracking para saber si ha llegado de alguno o no.
 - o En caso de tener que enviar el ACK, se realiza a través de un método del propio socket.io, de la siguiente forma:

```
socket.emit('realtime_emit_ack', { id: cb.id, payload: data });
```

- o cb.id es la propiedad “id” que trae el objeto “cb” del método “realtime_emit”.
- o payload: es cualquier objeto que esté esperando el método de retorno que se le proporcione en el ack.

Llamadas al backend:

Las siguientes llamadas al backend, se realizan una vez autenticado (tenemos las cookies en el navegador), todas son al mismo endpoint de la siguiente manera:

POST a la URL

<https://<servidor>.vozipcenter.com/u/0/v3/backend>

¡ojo!, ahora después del dominio hay una U (de Usuario) en vez de una L (de Login).

En el body, siempre mandamos:

```

{
  "method": "<nombre del método>",
  "args": null
}

```

Siendo "method" el nombre del método a llamar y args el objeto a enviar.

CAMBIO DE GRUPO/CALLERID

El grupo puede cambiar desde el servidor, al cambiarlo en la App móvil o en la web, por lo tanto, se deben sincronizar.

Si queremos modificar el grupo a través del que va a lanzar las llamadas, o modificar el callerid de dicho grupo, debemos llamar a un método del backend, el sistema enviará el evento "realtime_sync" a todos los dispositivos (nosotros incluidos), por lo tanto, no debemos modificar manualmente el valor del objeto __b2com_realtime, dado que será el evento (realtime_sync) disparado en el servidor quien lo actualice.

Cambiamos el grupo llamando a:

```
{
  "method": "agente/set_group",
  "args": {
    "group": "3",
    "callerid": ""
  }
}
```

Si el grupo no tiene array de callerid, lo mandamos vacío. En caso de que grupo traiga una lista de callerid's debemos proporcionar uno para realizar llamadas.

```
{
  "method": "agente/set_group",
  "args": {
    "group": "1",
    "callerid": "942049038"
  }
}
```

Cambio de estado de disponibilidad

La lista de estados, la obtenemos de __b2com_state.user.estados. Funciona de la misma forma que cambiar el grupo.

```
{
  "method": "agente/set_status",
  "args": {
    "status": 6
  }
}
```

CANAL DE VOZ:

A nivel de arquitectura, ya no se debe mantener un canal de voz establecido, sino que cada llamada establecida, realizará una llamada de voz a través de JsSip. Aún así, se mantiene la conexión SIP sobre Ws de jssip.

Utilizamos la librería JsSip v3.10.1 "jssip": "^3.10.1",

Puede descargarse el fichero jssip.min.js de la última reléase en:

<https://github.com/versatica/JsSIP/tags>

Inicializamos un único objeto UA (UserAgent) y lo arrancamos para establecer la comunicación WS:

```
const ua = new JsSIP.UA({
  sockets: [
    new JsSIP.WebSocketInterface(`wss://${window.location.host}/ws3`),
  ],
  register: false,
  uri: `sip:b2com_user@${window.location.host}`,
});
ua.on('connected', () => {
  ua.sendOptions('sip:b2com_user', '');

  window.addEventListener('beforeunload', () => {
    ua.stop();
  });
});
ua.start(); //Abre el WS.
```

La URL de acceso, tanto para la conexión como para las credenciales, debe corresponder con la del propio dominio utilizado (todo el sitio funciona a través de una única URL del backend).

Si el acceso es a premium3.vozipcenter.com la conexión se establece a:

wss://premium3.vozipcenter.com/wss

Y el URI de acceso a:

sip:b2com_user@premium3.vozipcenter.com

LLAMADAS DE VOZ SALIENTES

Para realizar una nueva llamada, llamamos al método:

```
{
  "method": "agente/phonecall",
  "args": {
    "numero": "310015140"
  }
}
```

Este método, nos devuelve información de la llamada que vamos a realizar:

```
{
  "id": "3041580",
  "contacto_nombre": "Fijo Jorge,JORGE,Jorge Leal,Juan JOS?",
  "contacto_id": "217397",
  "numero": "310015140",
  "modo": "saliente",
  "grupo_id": 1,
```



```

    "grupo_nombre": "grupo A",
    "geo": "942049038",
    "pais": "ES"
  }

```

De la información devuelta, nos interesa el “id”, este identificador único de llamada, es el que necesitamos para realizar la llamada por WebRTC a través de JsSip.

Para realizar la llamada con JsSip usamos:

```

const turnConfig = {
  iceServers: [
    {
      urls:
        `turn:${window.location.host}:19302?transport=tcp`,
      credential: 'pirSWYuhtGL2PZje',
      username: 'username',
    },
  ],
  iceTransportPolicy: 'relay',
};

const options = {
  extraHeaders: [`b2comID: ${__b2com_state.user.token}`],
  eventHandlers,
  mediaConstraints: { audio: true, video: false },
  pcConfig: turnConfig,
  mediaStream: audio,
};

const session = ua.call(`sip:m${call_id}`, options);

```

donde “call_id” es el “id” devuelto por el método “phonecall” anterior. Debemos pasar una letra **m** delante del id, para indicar que es una llamada saliente.

En el objeto **turnConfig**, debemos proporcionar nuevamente la url del servidor, que será quien haga relay de voz a través del puerto tcp 19302.

En el objeto **options**, debemos proporcionar:

- Un token único de usuario, se obtiene del objeto `__b2com_state.user.token`
- Audio, es el stream de audio local, obtenido a través de `getUserMedia`
- `eventHandlers` es un objeto que gestiona los distintos eventos de progreso de la llamada:

```

var eventHandlers = {
  progress: function (e) {

```

```

        console.log('call is in progress:', e);
    },
    failed: (e) => {
        console.log('call failed with cause: ', e, call_id);
    },
    ended: (e) => {
        console.log('call ended with cause: ', e, call_id);
    },
    confirmed: function (e) {
        console.log('call confirmed', e);
    }
};

```

Del objeto “sesión” devuelto por el método “call”, debemos escuchar el evento del track para obtener el audio remoto:

```

session.connection.addEventListener('track', (x) => {

    streamRemoto = x.streams[0];
});

```

Dicho stream debemos pasarlo a la propiedad srcObject de un objeto audio para escucharlo.

Cuando la llamada sea finalizada desde el lado remoto, nos llegará por un lado, el evento “ended” del objeto “eventHandler” para indicar que se ha finalizado el canal de audio y es el encargado de finalizar el acceso al micrófono, actualizar el dialpad.

Y simultáneamente llegará un evento realtime_emit/call/insert_ultima_llamada que nos enviará la información de la llamada ya finalizada:

```

{
  "store": "call",
  "method": "insert_ultima_llamada",
  "payload": {
    "id": 3041582,
    "modo": "saliente",
    "contacto_id": 217397,
    "contacto_nombre": "Fijo Jorge",
    "numero": "310015140",
    "grupo_nombre": "grupo A",
    "grupo_id": 1,
    "geo": "942049038",
    "inicio": "2024-07-09T11:25:05Z",
    "contestada": "2024-07-09T11:25:08Z",
    "duracion": 11,
    "fin": "2024-07-09T11:25:19Z",
    "status": "Atendida",
    "unread": 0
  },
  "cb": {
    "id": null
  }
}

```

Para finalizar la llamada por nuestro lado, simplemente finalizamos el canal de voz, llamando al método “terminate” del objeto “session” devuelto por el “ua.call”:

```
session.terminate({
  status_code: 487,
});
```

Y de la misma forma que en el caso anterior, recibiremos un evento `realtime_emit/call/insert_ultima_llamada` con la información correspondiente.

Para las operaciones propias de telefonía, usamos las proporcionadas por JsSip:

- `session.hold()`
- `session.unhold()`
- `session.mute()`
- `session.unmute()`

Si realizamos una llamada y el otro extremo esta ocupado, recibiremos el evento “failed” del objeto “eventHandlers” y se disparará el “insert_ultima_llamada” como en los casos anteriores.

LLAMADAS DE VOZ ENTRANTES

La notificación de llamadas de voz entrantes, se realiza a través de un `realtime_emit`.

Todos los navegadores del usuario son notificados, pero solo uno de ellos contesta, el resto de navegadores será notificados para “ocultar” la notificación de llamada, dado que se ha contestado en otro.

El método para notificar la llamada entrante es `realtime_emit/call/call_incoming`:

```
42["realtime_emit", {store: "call", method: "call_incoming",...}]
0: "realtime_emit"
▼ 1: {store: "call", method: "call_incoming",...}
  ▶ cb: {id: null}
  method: "call_incoming"
  ▼ payload: {modo: "entrante", geo: "000000000", id: "3041591", grupo_id: 1, grupo_nombre: "grupo A",...}
    contacto_id: "217398"
    contacto_nombre: ""
    geo: "000000000"
    grupo_id: 1
    grupo_nombre: "grupo A"
    id: "3041591"
    modo: "entrante"
    numero: "140"
    pais: "ES"
    store: "call"
```

El payload nos devuelve la información de la llamada.

En el momento que otro navegador conteste la llamada, o hayan colgado en origen, se lanza el evento `realtime_emit/call/call_finish`, que nos proporciona el mismo ID de llamada del evento anterior.

```

42["realtime_emit", {store: "call", method: "call_finish", payload: {id: "3041591"}, cb: {id: null}}]
  0: "realtime_emit"
  ▼ 1: {store: "call", method: "call_finish", payload: {id: "3041591"}, cb: {id: null}}
    ▶ cb: {id: null}
      method: "call_finish"
    ▼ payload: {id: "3041591"}
      id: "3041591"
      store: "call"

```

Una vez notificamos al usuario de la llamada, tenemos la opción de contestar, o rechazar.

Para rechazar la llamada, llamamos al método agente/reject, proporcionando el ID de llamada.

```

{
  "method": "agente/reject",
  "args": {
    "id": 3041592
  }
}

```

Para contestar la llamada, primero debemos llamar al método del backend agente/answer, que será quien asigne un ganador, en caso de que multiples usuario/dispositivos contesten simultáneamente la llamada:

```

{
  "method": "agente/answer",
  "args": {
    "id": 3041593,
    "unique": "b051d1c78a5749cda665c77a18b2e833"
  }
}

```

En valor de “unique”, es un valor aleatorio que debemos generar para cada “ventana” abierta del mismo usuario, sirve para el marcador en el futuro, donde las llamadas se contestan automáticamente.

El método answer solo nos devuelve:

```

{
  "id": "3041593"
}

```

Con ese ID, debemos realizar una llamada con JsSip, igual que para las llamadas salientes anteriores, pero en vez de llamar a [sip:m{id}](#), llamaremos a: [sip:e{id}](#)

Para finalizar la llamada entrante, terminamos la sesión, como hacíamos con la saliente.

CONFERENCIAS:

Las conferencias se realizan completamente desde el lado del cliente.

Para ello, una vez lanzadas 2 llamadas salientes, procedemos a unir las entre ellas.

Una vez establecida y contestada una llamada (tanto entrante como saliente), le damos al botón de “Añadir”, pausamos la primera llamada y establecemos una nueva con el mismo procedimiento que la original.

Guardamos las “sesiones” devueltas por el método:

```
const session = ua.call(`sip:m${call_id}`, options);
```

Para interactuar individualmente con cada llamada.

En el momento que tengamos 2 llamadas establecidas (ya contestadas), podemos darla a “fusionar”, en ese momento quedamos en conferencia con ellas 2.

El navegador es el encargado de transmitir el audio entre llamadas, si se cierra el navegador, las llamadas se finalizan.

Para ello, genero por cada llamada, un stream de audio saliente, donde le envío mi micrófono para la recepción del resto de llamadas. De forma que cada llamada me escuchará a mí, junto con el audio que recibo del resto de llamadas.

El script que utilizamos actualmente es el siguiente, al que se le pasa el ARRAY con todas las llamadas a fusionar.

No se realiza ninguna interacción con el backend para la conferencia de llamadas, se produce totalmente desde el navegador.

```

static conf(calls) {
  const context = new AudioContext();

  //Por cada llamada.
  for (const call_id of calls) {
    const session = sesiones[call_id];

    const mixedOutput = context.createMediaStreamDestination(); // <
    Audio del resto de llamadas + el mio.

    for (const call_id2 of calls) {
      const session2 = sesiones[call_id2];
      if (session2 == null || session2.id == session.id) {
        continue;
      }

      const receivers = session2.connection.getReceivers();

      receivers.forEach((receiver) => {
        const sourceStream = context.createMediaStreamSource(
          new MediaStream([receiver.track])
        );
        sourceStream.connect(mixedOutput);
      });
    }

    const sourceStream = context.createMediaStreamSource(
      VoipService.localAudio[call_id]
    );
    sourceStream.connect(mixedOutput);

    const senders2 = session.connection.getSenders();
    senders2[0].replaceTrack(mixedOutput.stream.getTracks()[0]);
  }
}

```

MARCADOR:

La única funcionalidad a implementar, es el “descuelgue automático”, por este proceso, cuando una llamada es contestada por un usuario remoto, instantáneamente nos entra y descuelga una llamada.

Dado que podemos tener múltiples navegadores abiertos, el sistema elegirá uno de ellos para contestar la llamada.

Cada navegador, debe generar un ID único en el momento de iniciarse por primera vez.

Dicho ID se regenera cada vez que se refresca la página y no debe almacenarse ni en cache ni en localStorage, simplemente define la ejecución única de cada instancia de la página.

Puede usarse cualquier método para generarlo, nosotros estamos usando este:

```
const generateUniqueId = () => {
  // Crea un array de bytes usando crypto.getRandomValues
  let array = new Uint8Array(16);
  crypto.getRandomValues(array);

  // Convierte los valores a una cadena hexadecimal
  let hexString = '';
  for (let i = 0; i < array.length; i++) {
    hexString += (array[i] < 16 ? '0' : '') + array[i].toString(16);
  }

  // Devuelve la cadena hexadecimal generada
  return hexString;
};

const unique_id = generateUniqueId();
```

Lo primero que llegará al navegador, con una llamada del marcador, es una llamada del método "marcador_incoming" en el modulo “call” a través del evento del socket.io “realtime_emit”.

Este método, recibe un objeto con información de la llamada (que ahora no vamos a usar para nada) y una función de callback, a la que debemos llamar pasando el id único generado al inicio de la página.

```
async marcador_incoming(call, cb) {
  cb({ unique: unique_id });
},
```

Este método se ejecuta en todas las instancias abiertas del navegador, el navegador que primero responda, será el elegido para responder la llamada.

El siguiente método que nos llegará, también a través del “realtime_emit” en el módulo “call” es “marcador_answer”.

Este es el método donde nos llega toda la información de la llamada.

¡OJO!, solo debemos contestar la llamada si el ID único que nos llega corresponde con el nuestro, sino debemos ignorarla.

De esta forma, solo el navegador “ganador”, será el encargado de responder la llamada.

Nuestro método de responder la llamada es:

```
async marcador_answer({ unique, call }) {
  if (unique !== unique_id) {
    //No es para mí!!!
    console.log('Llamada de otra sesión, la ignoro...');
    return;
  }
  console.log('LLAMADA ACEPTADA: ', call);
  this.register_call(call);
  this.answer_call(call.id);
},
```

El mecanismo de responder la llamada es el mismo que para una llamada entrante, es decir, llamar al backend al método agente/answer utilizando el id de llamada recibido y posteriormente una llamada de voz a [sip:e{xxx}](#) para establecer la comunicación.

LLAMADAS SALIENTES VÍA API:

Llamando a una API del servidor, podemos lanzar una llamada saliente a un agente determinado.

Para permitirlo, en el agente debemos usar una técnica similar a las llamadas de marcador.

Primero recibimos una llamada al método “api_make_new_call_incoming” al que le devolvemos nuestro ID único de navegador:

```
async api_make_new_call_incoming(call, cb) {
  cb({ unique: unique_id });
},
```

Posteriormente recibiremos una llamada el método “api_make_new_call” que debemos ignorar si no es para nosotros y lo único que realiza es una nueva llamada saliente, al número de teléfono proporcionado por el método.

```
async api_make_new_call({ unique, remote }) {
  if (unique !== unique_id) {
    return;
  }

  this.makeCall(remote);
},
```